

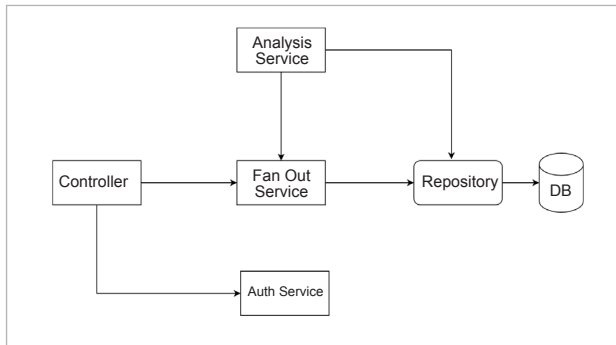


Figure 2: Sample architecture

**Services:** The domain or business logic resides in the service layer. It takes inputs from the controller layer and processes it based on the requirements of the application at a business level. This may include something as complex as initiating a banking transaction or a simple follow feature in a social media application. The service layer talks to the data layer to create, read, update, or delete data.

Services are defined by the `@Service` annotation on top of a class in Spring. A service can make use of other services, components, repositories and other dependencies to get something done.

In our example system, we have an auth service that is called by the controller to ensure that the request being made is authenticated. There are many ways to achieve this. One of them is using JWT authentication, which is quite popular in microservices based systems. There are many libraries like Spring Security that make it easy and systematic for us.

The Fan Out service or the timeline service is responsible for talking to the repository layer to read or persist data. It calls the analysis service to perform some data analysis to understand the user's taste and preferences in order to suggest better ads or articles to them.

**Repositories:** To access data, Spring has repository interfaces that talk to the database either with a data mapper like JPA's entity manager or raw JDBC connections. Spring, on the other hand, abstracts this out and provides convenience repositories for doing typical CRUD activities.

Repositories are annotated with the `@Repository` on top of the class or interface. A typical SQL based system can leverage Spring's JPA repository without writing too much code. This is done by extending the `JpaRepository` interface.

A JPA repository operates on domain objects known as entities. These are plain old Java objects that have a business value and have a direct mapping as a table in

a database. For example, a user can be an entity that has a mapping to a table in the SQL database.

This is defined with the `@Entity` annotation on a data class:

```
@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String firstname;
    private String lastname;
}
```

The `@Id` annotation says that the field ID should be used to uniquely identify the entity. And the `@GeneratedValue(strategy = GenerationType.AUTO)` annotation says that it will be automatically generated during persistence for the first time by Spring or the database, and doesn't have to be handled by the user.

This entity can then be used with a JPA repository:

```
@Repository
public interface UserRepository extends Repository<User,
Long> {

    List<User> findByEmailAddressAndLastname(String
emailAddress, String lastname);
}
```

The above code automatically creates a mapping between the user entity and the database, and generates convenience CRUD methods like `save()` and `findById()` without any more code. In fact, the method declared in the above example interface doesn't even require the user to provide an implementation for the method. Spring does it for you!

Although this is convenient, there are times when we do need to take control of the database logic. We can do this in two ways.

**1. By using the `@Query` annotation:** This allows us to write JPQL queries:

```
@Query("select u from User u where u.firstname LIKE ?1")
List<User> findUsersWithNamesLike(String name);
```

We can also use native queries like this:

```
@Query("select * FROM USERS WHERE FirstName LIKE ?1",
nativeQuery = true)
List<User> findUsersWithNamesLike(String name);
```